

Memory Efficient Doubly Linked List
–Insert At Position and Time Complexity
–Part 2

***When if(next $\neq$ NULL) $\rightarrow$ Condition is False. When we enter the
last node***

let , pos = 5

```
while (curr != nullptr && i < pos - 1)
{
    next = XOR(prev, curr->npx);
    prev = curr;
    curr = next;
    i++;
}
```

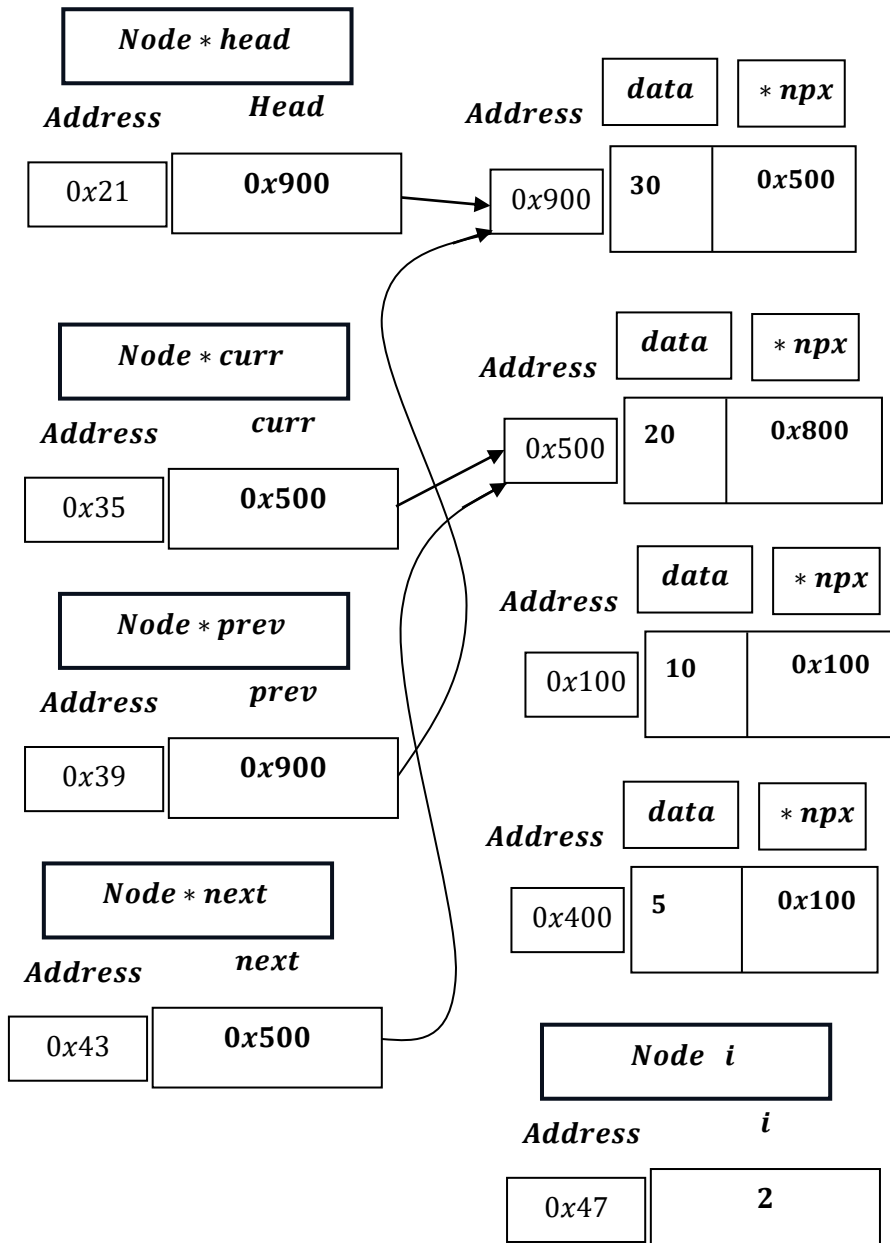
$curr(0x900) \neq 0x000$ And $i < (5 - 1 = 4) \Rightarrow 1 < 4$ is true:

$next = XOR(0x000, curr \rightarrow npx \Rightarrow 0x500) = 0x500$.

$prev = curr = 0x900$;

$curr = next = 0x500$;

$i = i + 1 = 1 + 1 = 2$;



$curr(0x500) \neq 0x000$ And $i < (5 - 1 = 4) \Rightarrow 2 < 4$ is true:

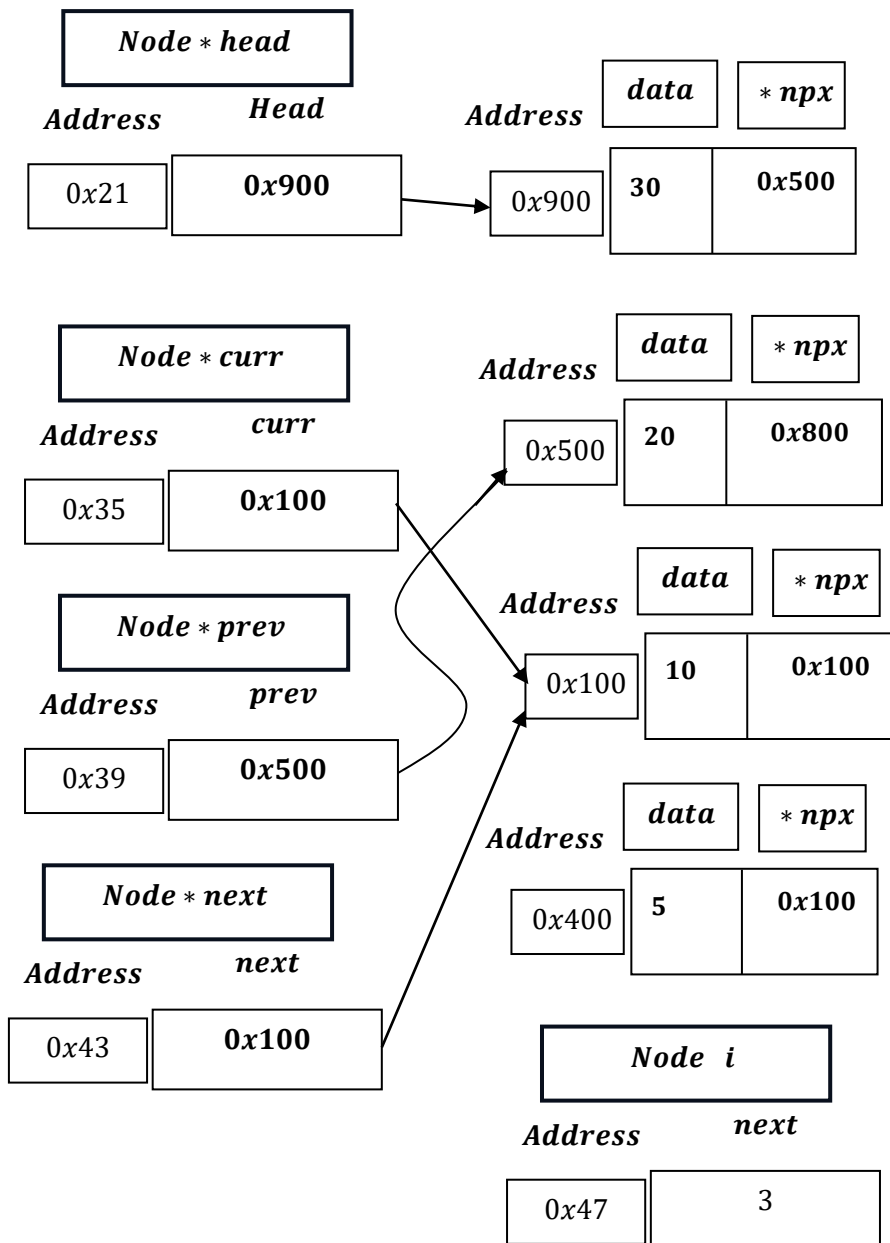
$next = XOR(0x900, curr \rightarrow npx \Rightarrow 0x800)$

$= 1001_2 \oplus 1000_2 = 0001_2 = 0x100$

$prev = curr = 0x500;$

$curr = next = 0x100;$

$i = i + 1 = 2 + 1 = 3;$



$curr(0x100) \neq 0x000$ And $i < (5 - 1 = 4) \Rightarrow 3 < 4$ is true:

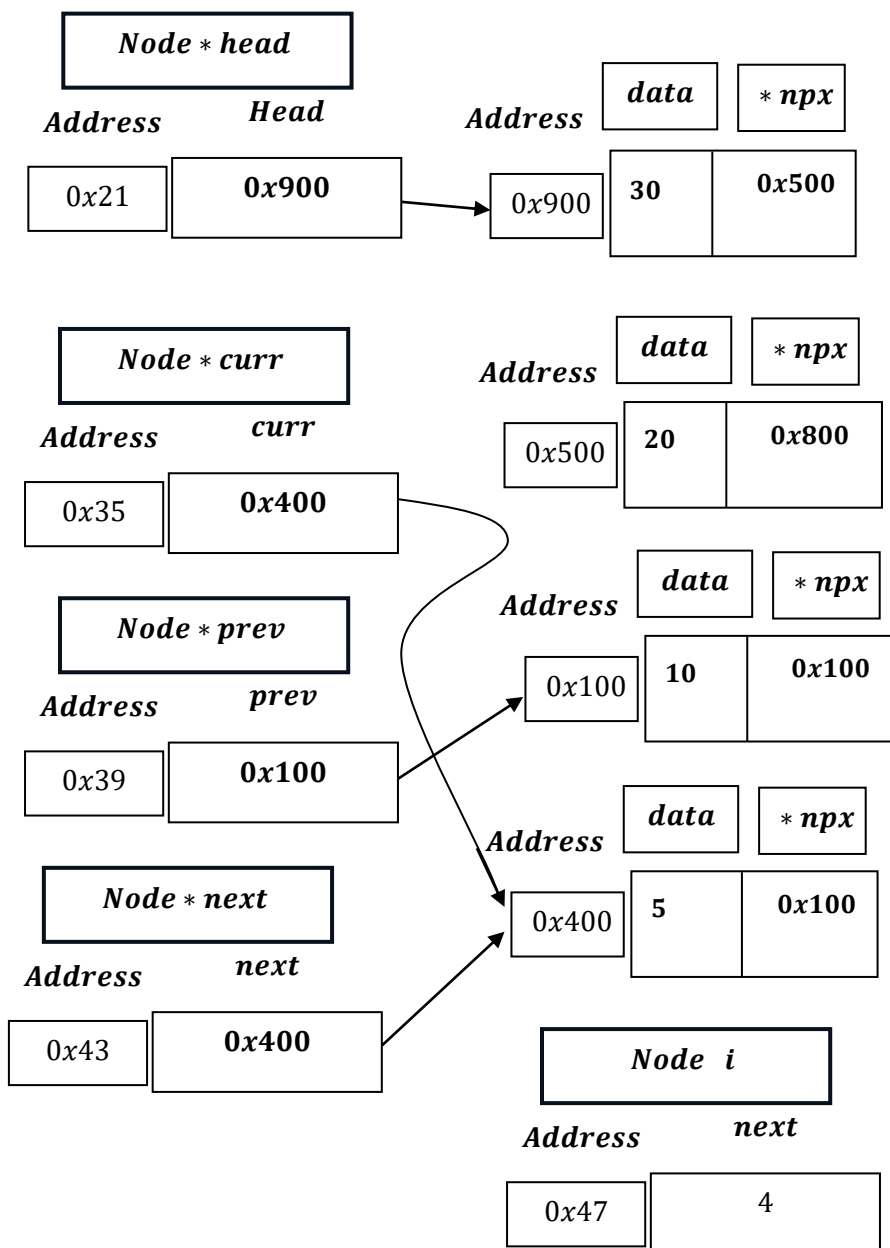
$next = XOR(0x500, curr \rightarrow npx \Rightarrow 0x100)$

$= 0101_2 \oplus 0001_2 = 0100_2 = 4_{10} = 0x400$

$prev = curr = 0x100;$

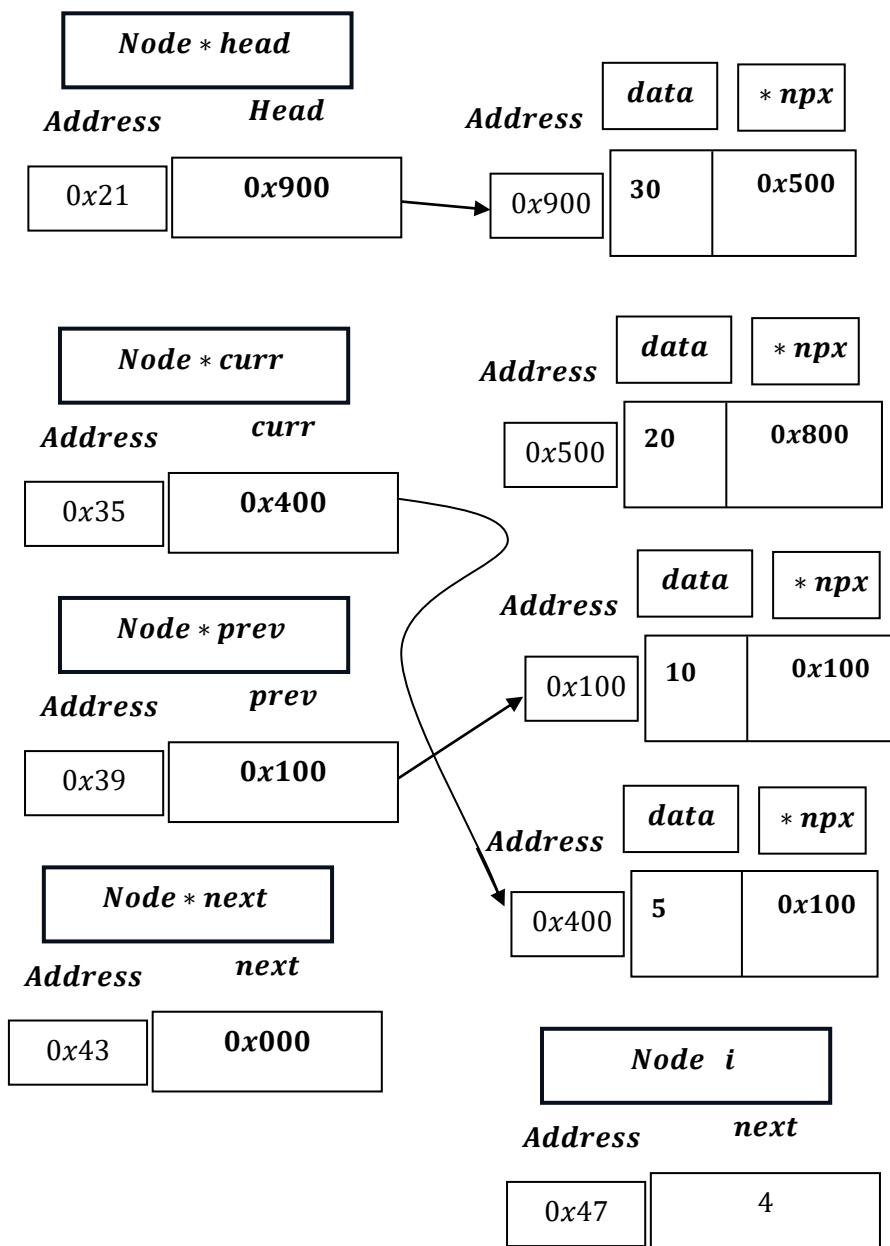
$curr = next = 0x400;$

$i = i + 1 = 3 + 1 = 4;$



$curr(0x400) \neq 0x000$ And $i < (5 - 1 = 4) \Rightarrow 4 < 4$ is false. Hence , loop exits.

```
next = XOR(prev, curr->npx);
next = XOR(0x100, 0x100) = 00012  $\oplus$  00012 = 00002 = 010 = 0x000
```



```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ *The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.*

→ *The size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.*

→ *The size of the Node structure itself is determined by the sum of the sizes of its members:*

→ *data(an integer) is typically 4 bytes.*

→ *npx(a pointer to another Node) is typically 8 bytes.*

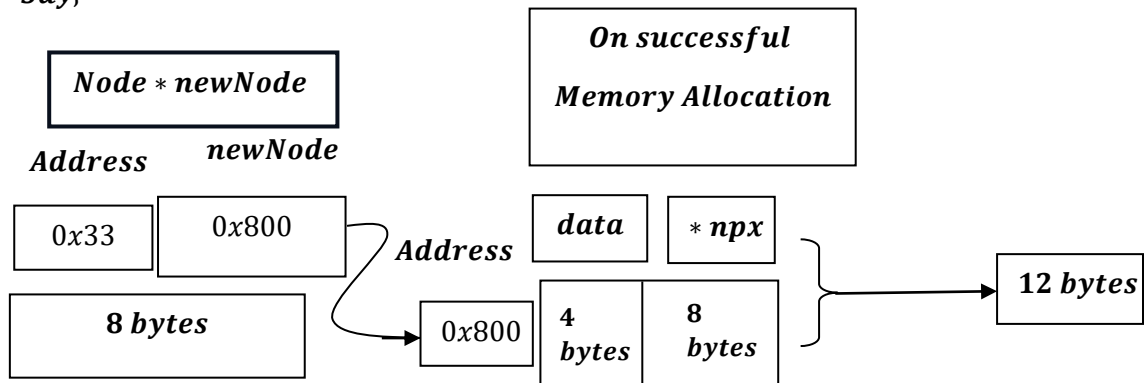
→ *Therefore ,the size of the Node structure is typically 12 bytes(4 bytes for data + 8 bytes for npx) .*

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

What does it mean?

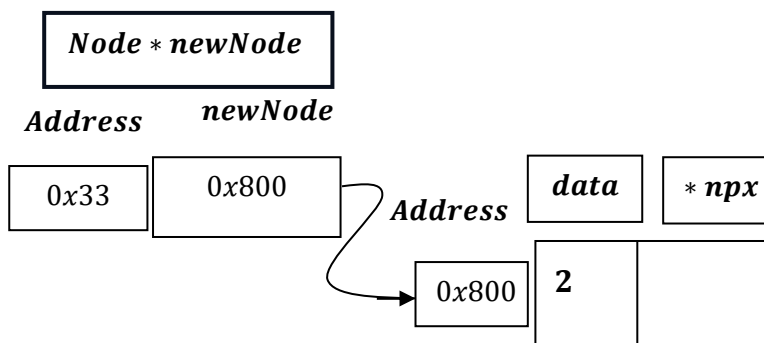
→ *newNode will try to allocate memory for data i.e. 4 bytes ,npx pointer variable i.e. typically 8 bytes, i.e. total 12 bytes, as discussed earlier ,the size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.*

Say,



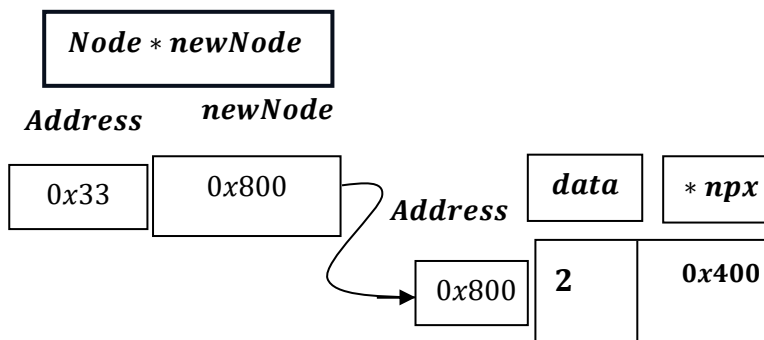
```
newNode->data = data;
```

Say, $data = 2$. Then, $newNode \rightarrow data = data = 2$.

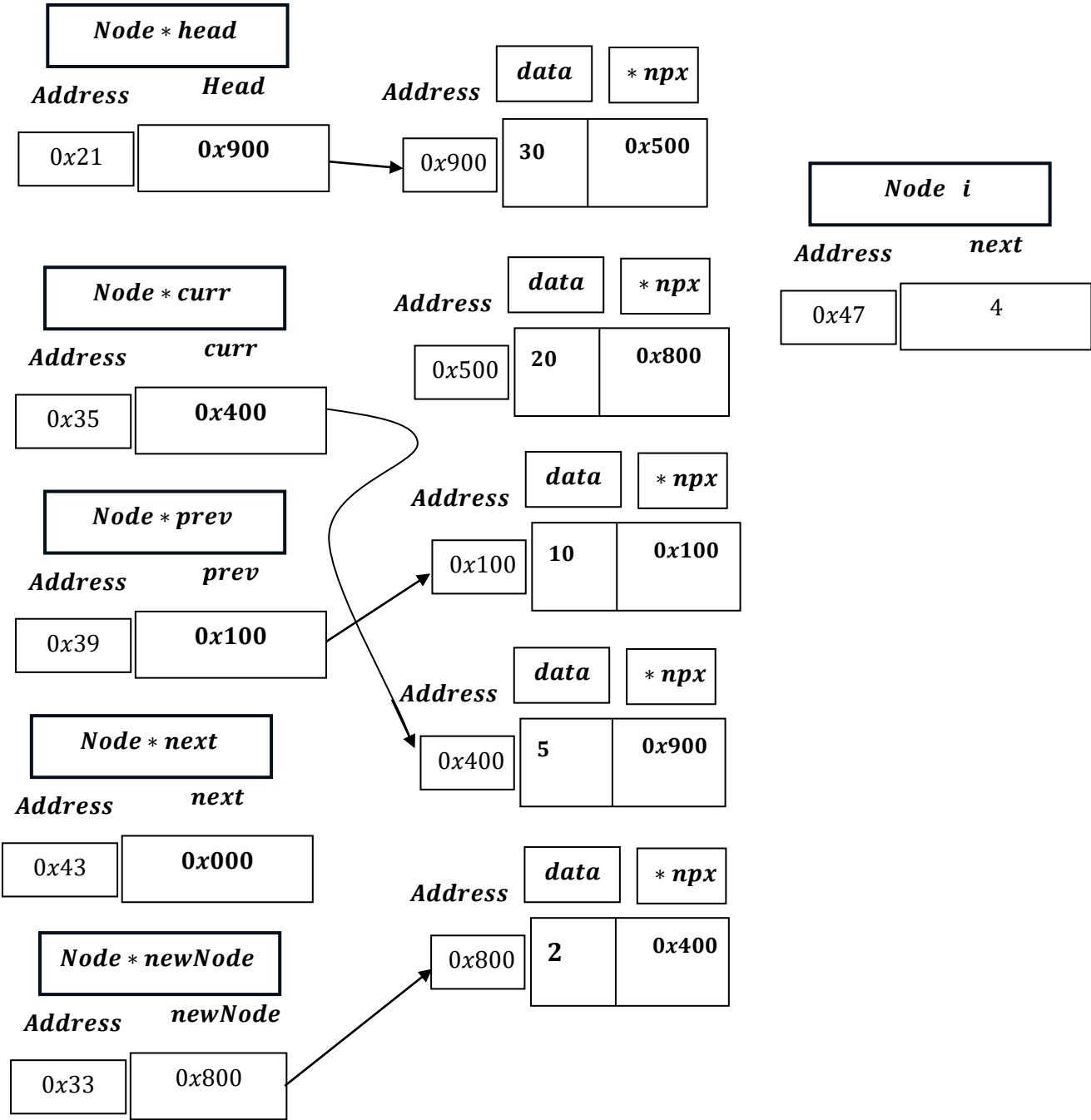


```
newNode->npx = XOR(curr, next);
```

Say, $newNode \rightarrow npx = XOR(0x400, 0x000) = 0x400$



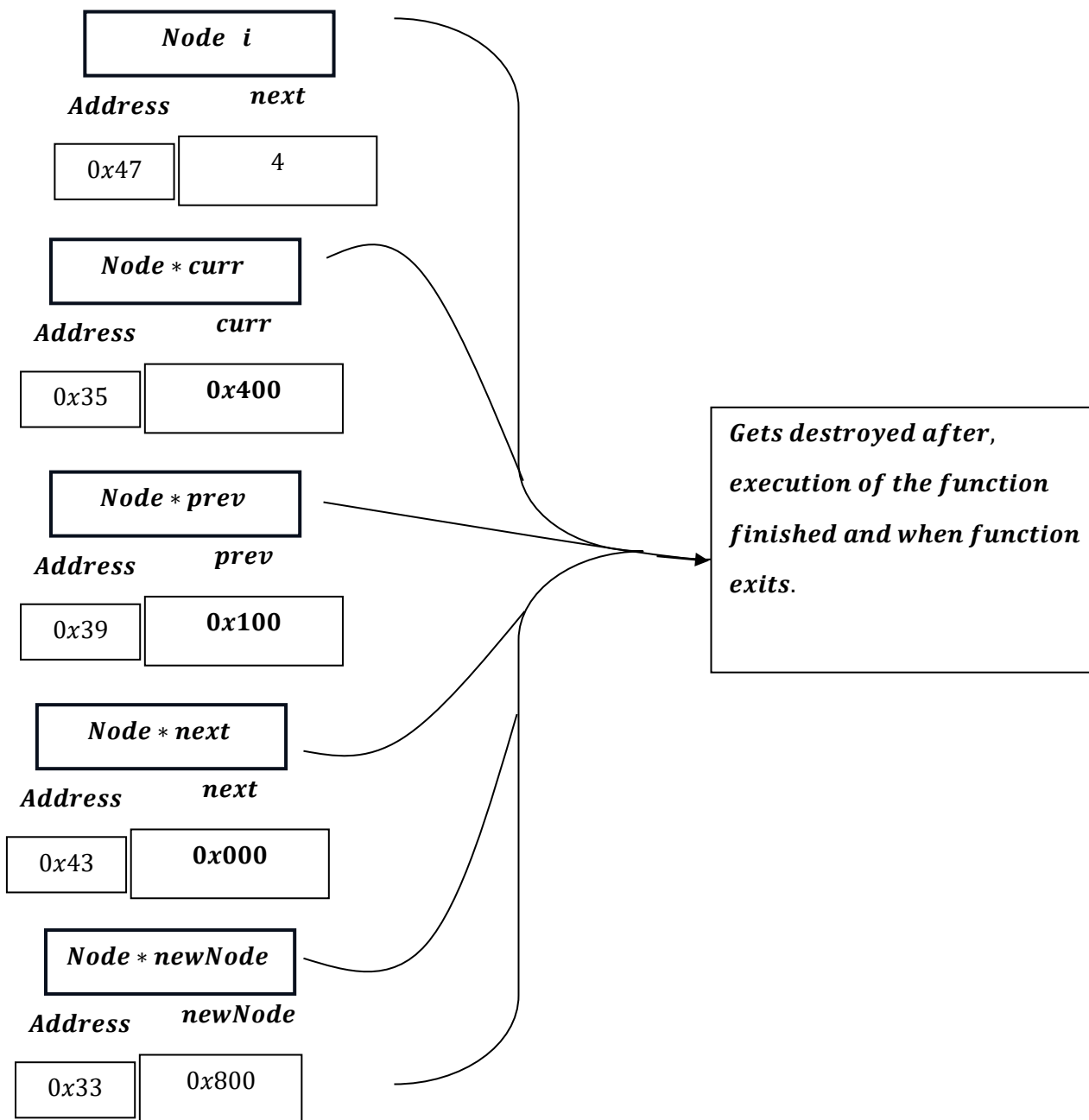
```
curr->npx = XOR(prev, newNode);  
curr → npx = XOR(0x100, 0x800) = 00012 ⊕ 10002 = 10012 = 910 = 0x900
```



Now, we see that, $next = 0x000$ i.e. $NULLPTR$ hence ,

When $if(next \neq NULL) \rightarrow$ Condition is False.

As , functions ends:



And we are left with:

